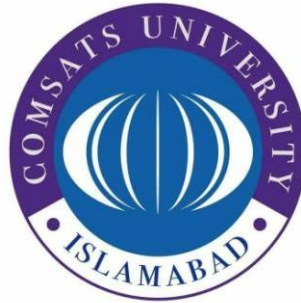


COMSATS University Islamabad

Attock Campus



Department Of Computer Science

<i>Course</i>	<i>Information Security</i>
<i>Instructor</i>	<i>Ms. Ambreen Gul</i>
<i>Lab Assignment</i>	<i>04</i>

Student Details

<i>Registration No.</i>	<i>Name</i>
<i>FA24-BSE-008</i>	<i>Muhammad Afnan</i>

```

1  import hashlib
2
3  # TASK 1: SIMPLE RSA IMPLEMENTATION (WITHOUT LIBRARIES)
4
5  # Function to calculate GCD
6  def gcd(a, b):
7      while b != 0:
8          a, b = b, a % b
9      return a
10
11
12  # Function to find modular inverse using Extended Euclidean Algorithm
13  def mod_inverse(e, phi):
14      d_old, d_new = 0, 1
15      r_old, r_new = phi, e
16
17      while r_new != 0:
18          quotient = r_old // r_new
19          r_old, r_new = r_new, r_old - quotient * r_new
20          d_old, d_new = d_new, d_old - quotient * d_new
21
22      if r_old > 1:
23          return None # inverse doesn't exist
24      if d_old < 0:
25          d_old += phi
26
27      return d_old
28
29
30  # RSA Key Generation using small primes
31  def generate_keys(p, q):
32      n = p * q
33      phi = (p - 1) * (q - 1)
34
35      # Choose public key exponent e
36      e = 3
37      while gcd(e, phi) != 1:
38          e += 2
39
40      # Calculate private key exponent d
41      d = mod_inverse(e, phi)
42
43      return (e, n), (d, n)
44
45
46  # RSA Encryption (public key)
47  def rsa_encrypt(message, public_key):
48      e, n = public_key
49      cipher = []
50
51      for char in message:
52          m = ord(char) # convert character to number
53          c = pow(m, e, n)
54          cipher.append(c)
55
56      return cipher
57
58
59  # RSA Decryption (private key)
60  def rsa_decrypt(cipher, private_key):
61      d, n = private_key
62      message = ""

```

```

64     for c in cipher:
65         m = pow(c, d, n)
66         message += chr(m)
67
68     return message
69
70
71 # TASK 2: DIGITAL SIGNATURE USING RSA
72
73 # Create hash using SHA256
74 def sha256_hash(message):
75     return hashlib.sha256(message.encode()).hexdigest()
76
77
78 # Sign the hash using RSA private key
79 def sign_message(message, private_key):
80     d, n = private_key
81     hash_value = sha256_hash(message)
82
83     # Convert hash hex into integer
84     hash_int = int(hash_value, 16)
85
86     # RSA signature: s = hash^d mod n
87     signature = pow(hash_int, d, n)
88
89     return signature
90
91
92 # Verify signature using RSA public key
93 def verify_signature(message, signature, public_key):
94     e, n = public_key

```

```

95     hash_value = sha256_hash(message)
96     hash_int = int(hash_value, 16)
97
98     # Verify: hash == signature^e mod n
99     verified_hash = pow(signature, e, n)
100
101     return hash_int % n == verified_hash
102
103
104 # MAIN PROGRAM
105
106 print("\n===== TASK 1: RSA ENCRYPTION/DECRYPTION =====")
107
108 # Small prime numbers
109 p = 61
110 q = 53
111
112 # Generate keys
113 public_key, private_key = generate_keys(p, q)
114
115 print("Public Key (e, n):", public_key)
116 print("Private Key (d, n):", private_key)
117
118 # Message for encryption
119 message = "Muhammad Afnan"
120
121 print("\nOriginal Message:", message)
122
123 # Encrypt
124 cipher_text = rsa_encrypt(message, public_key)
125 print("\nEncrypted Cipher Text:", cipher_text)

```

```

127 # Decrypt
128 decrypted_text = rsa_decrypt(cipher_text, private_key)
129 print("\nDecrypted Message:", decrypted_text)
130
131
132 print("\n===== TASK 2: DIGITAL SIGNATURE =====")
133
134 # Create digital signature
135 signature = sign_message(message, private_key)
136
137 print("Message:", message)
138 print("SHA256 Hash:", sha256_hash(message))
139 print("Digital Signature:", signature)
140
141 # Verify signature
142 is_valid = verify_signature(message, signature, public_key)
143 print("\nSignature Verification Result:", is_valid)
144
145 # Modify message to show verification fails
146 modified_message = message + "!"
147
148 print("\nModified Message:", modified_message)
149 print("Modified SHA256 Hash:", sha256_hash(modified_message))
150
151 is_valid_modified = verify_signature(modified_message, signature, public_key)
152 print("\nVerification After Modification:", is_valid_modified)
153
154 if is_valid and not is_valid_modified:
155     print("\nDigital Signature Successfully Verified and Tampering Detected!")
156 else:
157     print("\nSignature Verification Failed!")

```

OUTPUT:

```
PROBLEMS OUTPUT TERMINAL ... Python + - [ ] [ ] ... [ ] [ ] X

/AppData/Roaming/uv/python/cpython-3.14.4-windows-x86_64-none/python.exe "c:/Users/lenovo/OneDrive/ドキュメント/New folder/New folder/LabAssignment04.txt"

===== TASK 1: RSA ENCRYPTION/DECRYPTION =====

Public Key (e, n): (7, 3233)
Private Key (d, n): (1783, 3233)

Original Message: Muhammad Afnan

Encrypted Cipher Text: [3004, 1297, 3052, 1818, 597, 597, 1818, 2872, 2774, 1317, 576, 1544, 1818, 1544]

Decrypted Message: Muhammad Afnan

===== TASK 2: DIGITAL SIGNATURE =====

Message: Muhammad Afnan
SHA256 Hash: e3251a481715056898428af4d6f004ac3062dfbd839815b8c47b17f3df5b83be
Digital Signature: 1806

Signature Verification Result: True

Modified Message: Muhammad Afnan!
Modified SHA256 Hash: b477b39bfa76b3a4ec74e93567192149033e3a3169f360e76fcebdbc06159ec

Verification After Modification: False

Digital Signature Successfully Verified and Tampering Detected!
PS C:\Users\lenovo\OneDrive\ドキュメント\New folder\New folder>
```